Tensor-Transformer Variants are Surprisingly Performant

by **Logan Riggs** 12th Jan 2026

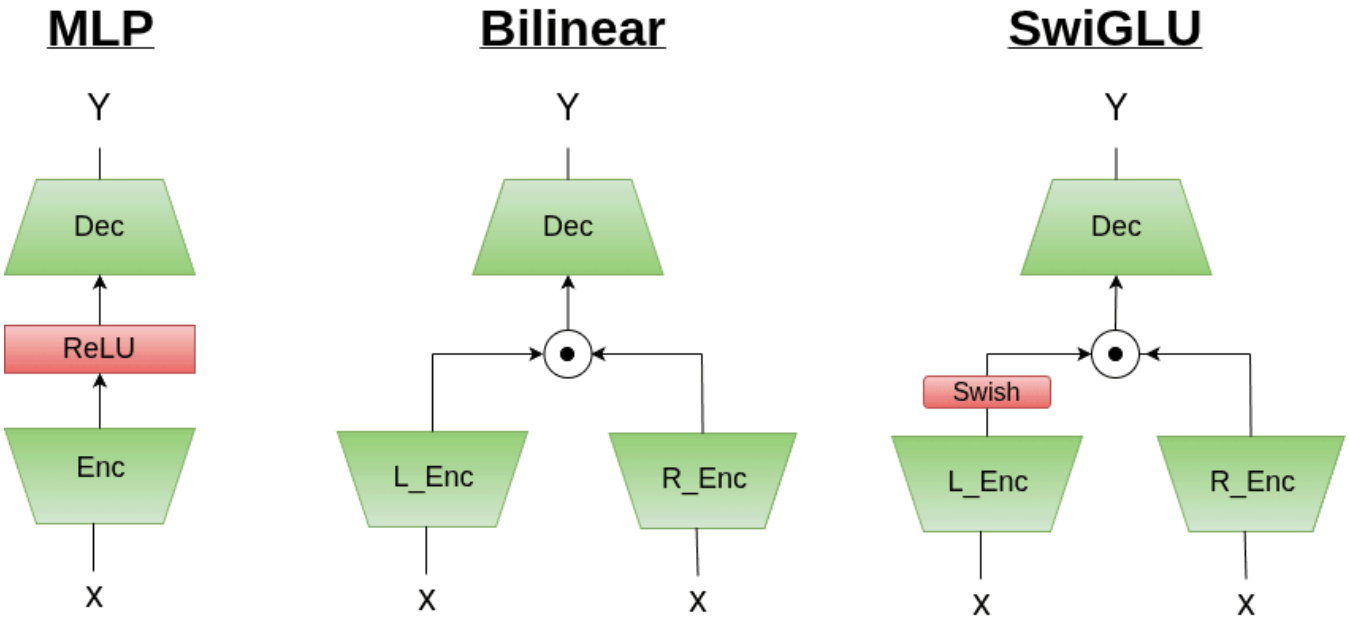
I've been researching tensor networks as a more interpretable architecture, but whenever I tell people this, they always ask "But is it any good?"

So I trained multiple 500M parameter LLMs on fineweb, showing the tensor variant needed ~4% more batches of data to match CE-loss.

There's a few caveats, so my personal estimate is around 15% worst to 10% better. Details below.

The Architecture

Replacing MLP w/ a Bilinear Layer



An MLP is a linear encoder, ReLU, then linear decoder.

$$MLP(x) = D(ReLU(E(x)))$$

A bilinear layer asks "what's better than one encoder? Two!"

$$Bilinear(x) = D(Lx \odot Rx)$$

Where $\odot$ means "element-wise multiply" eg

$$[1,2,3] \odot [1,2,3] = [1,4,9]$$

A SwiGLU Layer (Swish Gated Linear Unit) says "Let's add in nonlinearities"

$$SwiGLU(x) = D(swish(Lx) \odot Rx)$$

SwiGLU is a SOTA architecture & Bilinear is a tensor network.

Replacing Softmax Attn w/ Bilinear Attn

For a tensor network, we are only allowed polynomial nonlinearities. For attention, this means we need to replace softmax w/ a polynomial. The simplest is an element-wise

squaring of the attention pattern with itself.

$$Attn^2 = (xQK^\top x^\top)^2 = (xQK^\top x^\top) \odot (xQK^\top x^\top)$$

Notice how this is like computing the same attention pattern twice, then element-wise multiplying.

We can instead be slightly more expressive by training two sets of Q & K:

$$Bilinear\_Attn = (xQ_1K_1^\top x^\top) \odot (xQ_2K_2^\top x^\top)$$

Experiment & Results

I forked an older commit of modded-nanoGPT and trained four ~500M parameter LLMs (~GPT-medium size) on fineweb, switching out Bilinear & SwiGLU for the MLP-component and Softmax Attn & Bilinear attn for Attention. I trained on CE loss, comparing when they reached a loss of 3.0.

	Softmax (500M)	Bi_attn (550M)
Bilinear	100.0%	95.9%
Swiglu	99.6%	96.8%

Here, using the Bilinear or SwiGLU layer were basically the same, but switching to bilinear attn came at cost (but a very small one. Though note the 10% more parameters for Bi_attn)

I initially ran experiments in the GPT-2 small range:

	Softmax (190M)	Bi_attn (204M)
Bilinear	92.2%	80.4%
Swiglu	100.0%	88.6%

Which were much worse for the tensor-variants.

One might think "Oh, the tensor-variants actually scale better?", but I think it's because I forked from modded-nanogpt who's hyperparams & architecture is overfitted to the GPT-2 size model for Softmax attention. You'd need to do a larger hyperparam sweep (& various architecture changes, yikes) to get a better idea!

Caveats:

(1) Softmax attention ran faster cause it has a CUDA kernel

Softmax attention ran much faster because I was using F.scaled_dot_product_attention() which uses a cuda kernel under the hood. How to adjust? I don't want to write my own custom CUDA kernel, so I adjusted by saying they ran just as fast per step. This isn't quite true for the reasons below

(2) Bilinear Attention can run much faster than Softmax Attn

Bilinear Attention is $O(seq \cdot d_h^3)$ vs $O(\text{seq}^2 \cdot d_h)$ for normal softmax, where d_h := the head_dimension (usually d_model/num of heads)

So Bilinear attention is more efficient when:

$$seq > d_h^2$$

For a seq length of 1M & d_h of 128 (from deepseek-v3):

$$1e6 > 1.6e4$$

In other words, bilinear attention is more efficient computationally in this case when sequence length is > 1,600.

[More details & proof in appendix B°]

As a quick aside, we're gaining on computational efficiency, but this does come at a cost of less expressivity (see Appendix C°)

But what about Flash Attention?

See appendix D°

(3) Bilinear Attention has more Parameters

It's not fair that bilinear attention has twice the number of Q & K matrices, so I tried a baseline of differential attention from the literature which claims to need ~38% fewer parameters or ~36% fewer tokens. But it performed very poorly in my case(ie 100% worse)! It could've been a scaling issue, hyper-parameters, or coding bug (but the implementation is simple, see my code here).

(4) This was the 2nd-Dumbest Tensor-Attn Variant

There are likely way more efficient tensor-attn variants that exist. The 2nd dumbest is this bilinear attention, where the dumbest is just the .square() (which is like bilinear attention, but w/ tied Q's & tied K's weights).

Overall, I'm thinking this tensor attention variant is 15% worse to 10% better than softmax attention.

Replication & Trained Models

Code here, majority of code is just train_gpt2.py

Trained models here.[1]

Future Work

There's many more experiments one could run (eg scaling laws), but I'm currently focusing on actually doing interp w/ these models.

(Also, I've already spent ~$500 on 8 H100's for all the experiments. Speaking of which, special thanks to Principles of Intelligence (formerly PIBBSS) for funding me at the time of this research!)

Path to Impact

Tensor networks might actually be a viable alternative to typical NNs! However, if scaling is way worse (say 50%), then I highly doubt they'll be deployed as a frontier model.

But suppose we can solve ambitious mech interp w/ tensor networks (debatable but I lean yes), then there are two regimes:

1. Low Reliability
2. High Reliability

For writing math proofs we can verify, it's fine to have low reliability because failure is cheap. For self-driving cars though, you really want high reliability!

So we can do distillation or just train smaller tensor networks that aren't as generally capable, but are *task specific*.

Having highly robust, task-specific AI sounds great. So great, it might actually make a lot of money for various tasks.

This could change the financial incentives away from investing in low reliability AGI and towards highly reliable task-AI.

Interp w/ Tensor Networks

The second question I get when I bring up Tensor Networks is how they're actually more interpretable.

Most work on tensor networks isn't concerned with both ambitious interp and viability; however, Thomas Dooms, Ward Gauderis et al have been cooking this year!

Bilinear Autoencoders - they find structure in models using a bilinear AEs. See example manifolds here.

Compositionality Unlocks Deep Interpretable Models - a stack of bilinear layers is performant, while enabling us to view the global structure across the whole tensor network (since you can compose them together), though be warned, lots of math and kind of hard to understand.

Bilinear MLPS Enable Weight-Based Mech Interp - *"Bilinear MLPs can be fully expressed in terms of linear operations using a third-order tensor, allowing flexible analysis of the weights. Analyzing the spectra of bilinear MLP weights using eigendecomposition reveals interpretable low-rank structure across toy tasks, image classification, and language modeling. We use this understanding to craft adversarial examples, uncover overfitting, and identify small language model circuits directly from the weights alone."*

[And if I understand correctly, Transluce linearized their LLM per datapoint (making it a tensor network, but, again, only for that datapoint) to improve attribution.]

But I really believe this is just the tip of the iceberg. Tensor networks have a lot of useful properties that make them amenable to analytic tools. In short:

1. They can compose to show structure across the entire network.
2. They shift our focus from activations & data to directly interpreting the weights (which apply to the *entire* data distribution)
3. They are scale-invariant, meaning all circuits within the model don't change if the input changes scale. In other words, if you have a direction in layer 1, it will affect the exact same components downstream in the exact same ratios, regardless of how much you scale that direction.

As well as some forthcoming work from them that I'm (very) excited to see released!

As for me, I'm currently working on circuits applied to tensor networks. Do feel free to reach out to me here or on discord (# loganriggs) if you're interested in this research direction!

Special thanks to Thomas Dooms for reviewing an earlier draft of this post.

Appendix A: Noam Shazeer's 2020 paper:

In Noam Shazeer's 2020 paper, he trains these different architectures on a span filling task on the C4 dataset, showing their log-perplexity loss.

Training Steps	65,536	524,288
$\mathrm{FFN}_{\mathrm{ReLU}}(baseline)$	1.997 (0.005)	1.677
$\mathrm{FFN}_{\mathrm{GELU}}$	1.983 (0.005)	1.679
$\mathrm{FFN}_{\mathrm{Swish}}$	1.994 (0.003)	1.683
$\mathrm{FFN}_{\mathrm{GLU}}$	1.982 (0.006)	1.663
$\mathrm{FFN}_{\mathrm{Bilinear}}$	1.960 (0.005)	1.648
$\mathrm{FFN}_{\mathrm{GEGLU}}$	**1.942** (0.004)	**1.633**
$\mathrm{FFN}_{\mathrm{SwiGLU}}$	**1.944** (0.010)	**1.636**
$\mathrm{FFN}_{\mathrm{ReGLU}}$	1.953 (0.003)	1.645

Where the Bilinear layer does quite well! Even beating GLU (which should be called SiGLU for sigmoid GLU).

Appendix B: Scaling of Bilinear Attention

Proof from Claude Opus 4.5, edited & reviewed by me, w/ code verification here.

Our bilinear form is:

$$\underbrace{(xQ_1K_1^\top x^\top)}_{\text{seq}\times\text{seq}} \odot \underbrace{(xQ_2K_2^\top x^\top)}_{\text{seq}\times\text{seq}} \cdot v$$

where $\odot$ is the elementwise product. Naively this requires materializing a seq×seq matrix, which is $O(\text{seq}^2)$.

Defining:

$$q_1 = xQ_1 \in \mathbb{R}^{\text{seq}\times d_h}, \quad k_1 = xK_1 \in \mathbb{R}^{\text{seq}\times d_h} q_2 = xQ_2 \in \mathbb{R}^{\text{seq}\times d_h}, \quad k_2 = xK_2 \in \mathbb{R}^{\text{seq}\times d_h}$$

where d_h is the hidden dimension of the attention head. So the attention patterns are:

$$A_1 = q_1 k_1^\top \in \mathbb{R}^{\text{seq}\times\text{seq}}, \quad A_2 = q_2 k_2^\top \in \mathbb{R}^{\text{seq}\times\text{seq}}$$

The Row-wise Khatri-Rao Product, $\circledast$, is defined row-by-row as the Kronecker (outer) product of each row:

$$(A \circledast B)_{i,:} = A_{i,:} \otimes B_{i,:}$$

For example:

$A_{i,:} = [a_1, a_2]$ and $B_{i,:} = [b_1, b_2, b_3]$, then:

$$(A \circledast B)_{i,:} = [a_1 b_1, a_1 b_2, a_1 b_3, a_2 b_1, a_2 b_2, a_2 b_3]$$

A key identity[2] is

$$(AB^\top) \odot (CD^\top) = (A \circledast C)(B \circledast D)^\top$$

Using this identity:

$$A_1 \odot A_2 = (q_1 k_1^\top) \odot (q_2 k_2^\top) = (q_1 \circledast q_2)(k_1 \circledast k_2)^\top$$

Define:

$$\tilde{Q} = q_1 \circledast q_2 \in \mathbb{R}^{\text{seq}\times d_h^2} \tilde{K} = k_1 \circledast k_2 \in \mathbb{R}^{\text{seq}\times d_h^2}$$

So to compute $\tilde{Q}$ we're taking the row-wise outer product of the hidden dimensions of q_1 & q_2, repeating this for every sequence position (later, we'll mix across sequences when combining $\tilde{Q}$ & $\tilde{K}$).

Now the output becomes:

$$(A_1 \odot A_2) \cdot v = \tilde{Q}\tilde{K}^\top v$$

Everything here is just matrices/vectors being matrix-multiplied, so we have two choices on how to combine them:

1. Left to Right (Inefficient): $(\tilde{Q}\tilde{K}^\top)v$

- $\tilde{Q}\tilde{K}^\top$ is $(\text{seq} \times d_h^2) \cdot (d_h^2 \times \text{seq}) = O(\text{seq}^2 \cdot d_h^2)$
- Then multiply by v: $O(\text{seq}^2 \cdot d_h)$

So we're scaling quadratically in both seq-size & d_h

2. Right-to-Left (Efficient): $\tilde{Q}(\tilde{K}^\top v)$

- $\tilde{K}^\top v$ is $(d_h^2 \times \text{seq}) \cdot (\text{seq} \times d_h) = O(\text{seq} \cdot d_h^3)$
- Then $\tilde{Q} \cdot (\ldots)$ is $(\text{seq} \times d_h^2) \cdot (d_h^2 \times d_h) = O(\text{seq} \cdot d_h^3)$

So we're scaling linearly in seq-size & cubically in d_h

Appendix C: Bilinear Attention Expressivity

Softmax attention is more expressive. The max rank of the attention matrix for softmax is seq (full rank) whereas for bilinear attention, we're stuck at d_k^2. There's no free lunch, so while we're gaining a lot in computational efficiency, we're losing in expressivity.

In the kernel view, softmax approximates a Gaussian kernel, meaning it can approximate any continuous function, while bilinear attention is just a degree-2 polynomial kernel.

Appendix D: But what about Flash Attention?

Flash attention is about memory, not computation. It's purpose is to avoid materializing the full seq x seq matrix by splitting it up into tiles. You can't compute softmax over tiles since it's a property of entire rows, so they compute statistics on each tile which can be combined to ~compute softmax.

Tensor-Attention variants don't use softmax, so you don't need to do any clever tricks there (although the efficient bilinear attention method probably requires larger memory? I'll leave this as an exercise to the reader).

1. ^ The naming scheme goes as follows:

   eg Elriggs/gpt2-swiglu-18l-9h-1152embd

   is read sensibly as gpt2 w/ swiglu, 18 layers, 9 attn heads, & 1152 embedding dim. If attention isn't mentioned, it's assumed to be softmax.

   The rest of my naming scheme ended up being mixed up & non-ideal. I recommend going to each one's config.json if you're looking for a particular run.

   - squared_mlp is squaring the output of ReLU as typically done in modded-nano-gpt
   - bilinear :=bilinear layer
   - bilinear AND gated := SwiGLU (since a SwiGLU is a bilinear layer w/ a gate)
   - bilinear_attn := bilinear attention (but will be named "sqrd_attn" in the name. That's my mistake)

2. ^ Section 3, fourth equation down, citation is [16]

Mentioned in

37 When Are Two Networks the Same? Tensor Similarity for Mechanistic Interpretability
19 Black Boxes for Low-Stakes, Interpretable AI for High-Stakes

17 comments, sorted by top scoring

[–] **cat-state** 1mo ▼ 8 ▲ ✕ 0 ✓

You can add softmax and nonlinearities via using other semirings like log or max+

[–] **Logan Riggs** 23d ▼ 2 ▲ ✕ 0 ✓

Ah, that's a really cool connection; thanks!

This definitely works for maintaining contractions in a tensor network, but for our most recent work°, we rely on the standard semiring to be able to use linear algebra tools (eg cosine similarity). Switching to others would allow more standard non-linearities, but at the cost of interpretability (probably. I only really know linear algebra, lol).

[–] **Tao Lin** 6mo ▼ 6 ▲ ✕ 0 ✓

Redwood research did very similar experiments in 2022, but didn't publish about them. They are briefly mentioned in this podcast: https://blog.redwoodresearch.org/p/the-inaugural-redwood-research-podcast.

✚ 1

[–] **Logan Riggs** 3mo ▼ 5 ▲ ✕ 0 ✓

Thanks! It looks like they tried to interpret normal NNs by breaking them up into different order terms and used tensor diagrams as a tool. AFAIK, they didn't use tensor-transformers (I only ctrl-f-ed "tensor" and "bilinear", so could've missed it).

Though analyzing tensor transformers their way would also fail for the same reasons they brought up (ie exponential blow up of polynomial terms).

[–] **philip_b** 6mo ▼ 3 ▲ ✕ 0 ✓

Your bilinear attention layer is a bilinear function, but where the input (x) is copied and used as both inputs. Using those matrices Dec, L_Enc and R_enc, the way you show, is one particular way to parametrize the bilinear function. There are many other ways, the simplest of which would be to just use one tensor of shape [size-of-y, size-of-x, size-of-x]. I'm curious, why did you choose that particular parametrization?

Also, how did you initialize the model's weights? How do you initialize to prevent exploding gradients and similar problems?

I am curious about all this because my masters thesis was about a tensor network based alternative to CNNs.

[–] **Logan Riggs** 6mo ▼ 3 ▲ ✕ 0 ✓

A full 3rd order tensor is much larger, whereas this parametrization is the CP-decomposition form. This is the "official reason" when I'm really just building off Dooms et al. (I've never actually tried training the full tensor though!)

Re init: the init for modded gpt at that fork was kind of weird, but I'm pretty sure most standard inits prevent that. I am using RMSNorm which can be treated as a tensor network as well (I could maybe dm explanation, it's a forthcoming resource from Thomas). I'm also normalizing Q & K which isn't a tensor network, BUT compositionality is on a spectrum (maybe I am too). So this does mean a small portion of the model isn't a tensor network.

Ideally we can work around this!

🙏 1

[–] **HunterJay** 5mo* ▼ 2 ▲ ✕ 0 ✓

~~The fact that tensor-network architectures are scale-invariant seems underappreciated for useful steering. If my understanding is correct, it would mean that scaling the steering vector should cause the *same* pathways through the model to be activated, whereas without this we could be activating a totally different pathway, and get much less predictable behaviour.~~

Correction Below

[–] **Logan Riggs** 5mo ▼ 4 ▲ ✕ 0 ✓

I hadn't considered steering vectors before, but yes that's correct.

1

[–] **HunterJay** 5mo ▼ 1 ▲ ✕ -1 ✓

With more thinking, I was broadly wrong here:

- If you add a steering vector, it's not just scaling, so scale invariance doesn't make a difference.

- If you scale an existing activation vector which makes up the entirety of one of the layers, the only effect would be to change the absolute magnitudes going into the softmax (since scale invariance means the relative magnitude at each position is the same). That could have some minor effect -- changing the probability distribution to be sharper or flatter, but that's all.

- If you scale some existing activation which is *not* an entire layer, then it's no longer scale invariant anymore either, it's kind of like adding a steering vector with zero magnitude in the other dimensions.

There is still a weak advantage for steering vectors in a tensor network because the change is going to be smooth, rather than discrete (since we're not flipping gates on and off), but basically I was just confused here, sorry about that.

[–] **Logan Riggs** 5mo ▼ 4 ▲ ✕ 0 ✓

I'm confused on what you're referring to. Bilinear layers are scale invariant by linearity

$$Bilinear(ax) = a^2 Bilinear(x)$$

So x could be the input-token, a vector d (from the previous bilinear layer), or a steering vector added in, but it will still produce the same output vector (and affect the same hidden dims of the bilinear layer in the same proportions).

Another way to say this is that for:

$$y = Bilinear(ax)$$

The percentage of attribution of each weight in bilinear w/ respect to y is the same regardless of a, since to compute the percentage, you'd divide by the total so that cancels out scaling by a.

This also means that, solely from the weights, you can trace the computation done by injecting this steering vector.

[*Caveat: a bilinear layer computes interactions between two things. So you can compute the interaction between BOTH (1) the steering vector and itself and (2) the steering vector w/ the other directions d from previous layers. You CAN'T compute how it interacts w/ the input-token solely from the weights, because the weights don't include the input token. This is a bit of a trivial statement, but I don't want to overstate what you can get]

• • •

Overall, my main confusion w/ what you wrote is what an activation that is an entire layer or not an entire layer means.

[–] **HunterJay** 5mo ▼ 1 ▲ ✕ 0 ✓

You're correct, sorry for being confusing. Tracing through;

- My understanding of steering is that you can add a steering vector to an activation vector at some layer, which causes the model outputs to be 'steered' in that direction. I.e.:
  - Record layer n's activations when outputting "I am very happy", get vector h
  - Record layer n's activations when outputting "I am totally neutral", get vector q
  - Subtract q from h to get steering vector $s = h - q$, the difference between 'happy' and 'neutral' outputs.
  - Add αs to the activations at layer n to steer the model into acting more happy, where α is some scalar.
- The tensor network architecture is scale invariant, which (by my understanding) means that scaling the activation vector at any layer maintains the relative magnitude of the activations at any later layer.
- (Dumb) I thought that this meant that adding a steering vector of magnitude α and adding a steering vector of magnitude 2α would preserve the relative magnitude of the activations later in the network. That is, that scaling the steering vector would be scale invariant too. But that's not the case — we're changing the direction of the (activation vector + steering vector) when we increase the magnitude of the steering vector.

That's pretty much all I was trying to correct in my response. When I was talking about entire layer / not entire layer, I was just trying to say you can't pretend that adding a steering vector is actually just scaling the activation vector even if it is parallel in some dimensions. It's a trivial point I was just thinking through aloud. Like:

- If you have activation vector $v = [1, 2, 3, 4, 5]$
- You are scale invariant if you multiply by a scalar a: $av = a[1, 2, 3, 4, 5]$
- Which is the same as pointwise multiplication by the vector $u = [2, 2, 2, 2, 2]^\top$: $av = u \odot v$
- But you can't just say, "Well, I'm only going to scale part of vector u, and since it's scaling, that means it maintains scale invariance", because it's not just scaling, and that's a dumb thing to say — $u' = [2, 2, 2, 0, 0]^\top$, then $u' \odot v \neq av$ for any a.

So basically you can ignore that, I was just slowly thinking through the maths to come to trivial conclusions.

Your claim here is different and good, and points to another useful thing about bilinear layers. As far as I can tell — you are saying you can decompose the effect of the steering vector into separable terms purely from the weights, whereas with ReLU you can't do this because you don't know which gates will flip. Neat!

[–] **anaguma** 6mo ▼ 2 ▲ ✕ 0 ✓

> Softmax attention ran much faster because I was using F.scaled_dot_product_attention() which uses a cuda kernel under the hood. How to adjust? I don't want to write my own custom CUDA kernel, so I adjusted by saying they ran just as fast per step. This isn't quite true for the reasons below

Claude Opus 4.5 can make decent triton kernels these days; I'd recommend using that if attention is a bottleneck.

[–] **benjamin ar** 6mo ▼ 1 ▲ ✕ 0 ✓

> Tensor networks might actually be a viable alternative to typical NNs! However, if scaling is way worse (say 50%), then I highly doubt they'll be deployed as a frontier model.
>
> But suppose we can solve ambitious mech interp w/ tensor networks (debatable but I lean yes), then there are two regimes:
>
> 1. Low Reliability
> 2. High Reliability

Excited by the ambitious effort. Re: the impact argument above, I'm understanding your logic to be:

To completely replace NNs in frontier applications, scaling of TNs needs to be on par. Therefor, if we needed to replace them for TNs to be useful, we should test the scaling laws first. However, in a world where scaling is worse, TNs can still be useful by allowing for "ambitious mech interp" which would result in a High Reliability model. These two regimes aren't mutually exclusive.

Am I following the argument correctly?

♡ 1

[–] **Logan Riggs** 6mo ▼ 2 ▲ ✕ 0 ✓

Yep! But I do think the highest priority thing would be actually doing ambitious interp w/ this, although, if we had 100 people working on this (instead of ~4-5 full time?), a few working on the scaling laws would be good.

TNs are more amenable to optimizing exactly what we want in a mathematically precise way, so optimizing for this (to achieve ambitious mech interp) would incur an additional cost in capabilities, just fyi.

[–] **Roman Belaire** 6mo ▼ 1 ▲ ✕ 0 ✓

Disclaimer: I'm not very familiar with the ins and outs of tensor networks (so thanks for the reading list :D).

I would think that a composable dual encoder architecture would be able to hold more information than an MLP one, so it seems counterintuitive that the dual encoder requires *more* steps to achieve the

same cross-entropy. I'm sure this is in part due to the more complex loss function, so maybe there is some threshold on dataset size or model size above which the tensor variant achieves lower CE?

[–] **Logan Riggs** 6mo ▼ 3 ▲ ✕ 0 ✓

Just looking at Shazeer's paper (Appendix A)

Training Steps	65,536	524,288
$\mathrm{FFN}_{\mathrm{ReLU}}(baseline)$	1.997 (0.005)	1.677
$\mathrm{FFN}_{\mathrm{GELU}}$	1.983 (0.005)	1.679
$\mathrm{FFN}_{\mathrm{Swish}}$	1.994 (0.003)	1.683
$\mathrm{FFN}_{\mathrm{GLU}}$	1.982 (0.006)	1.663
$\mathrm{FFN}_{\mathrm{Bilinear}}$	1.960 (0.005)	1.648
$\mathrm{FFN}_{\mathrm{GEGLU}}$	**1.942** (0.004)	**1.633**
$\mathrm{FFN}_{\mathrm{SwiGLU}}$	**1.944** (0.010)	**1.636**
$\mathrm{FFN}_{\mathrm{ReGLU}}$	1.953 (0.003)	1.645

All of the GLU models performed better (lower is better) and the GLU models have a bilinear encoder (just w/ & w/o a sigmoid/GeLU/Swish/ReLU function). So in fact it does better (if this is what you meant by a dual encoder).

HOWEVER, we could have 3 encoders, or 100! This should store even more information, and would probably perform better *per step*, but would take up more GPU VRAM and/or take longer to compute each step.

In this post, though, I used *wall clock time* as a measure of training efficiency. Hand-wavy:

loss/step * time/step

(maybe it should be divided to make it loss/time?)

[–] **Roman Belaire** 6mo ▼ 1 ▲ ✕ 0 ✓

Ah, that makes more sense, thanks!

Also I agree with using loss/time as the measure of performance, since it's fairly straightforward to interpret (loss recovered per unit time). If I were reviewing this, I'd look for that.

For efficiency in practice, I think most ML papers look at FLOP/s since it is hardware agnostic. Maybe a good measure of efficiency here would be loss per FLOP per second? I haven't seen that used, but it might reflect how performance scales with computational speed.

Edit: Actually thinking about it, the test-time efficiency might be a better comparison, assuming the two scale within roughly the same complexity class. I think from a product perspective, speed for users is super (maybe the most) valuable.

Moderation Log

More from Logan Riggs

380	Hospitalization: A Review	Logan Riggs	9mo	21
37	When Are Two Networks the Same? Tens…	Logan Riggs, tdooms, Conflux, lw…	2mo	3
76	Consent-Based RL: Letting Models Endorse Their Own Traini…	Logan Riggs	3mo	6

View more

Curated and popular this week

411	Surprising facts about the slave trade ★	Joseph Miller	16h	35
329	A Mechanistic Explanation of Prompt Injection (and …	Charles Ye, Jasmine C.	6d	46
430	AI 2040: Plan A Ω	Daniel Kokotajlo, elifland, Thomas Larsen, romeo, …	6d	98